

Project 1: Socket Programming

6710451097 นายพรชัยวัฒน์ มานิตยกุล นาคแก้ว

Purpose

Sending messages through unreliable mediums and harsh conditions. *Such as radio communications in harsh weathers.*

Given these constraints, a Fire-And-Forget design is be much more preferable than relying on retransmissions or connection management in an environment where a significant package loss is expected.

Therefore, UDP is preferred over TCP as it introduces minimal protocol overhead and does not depend on continued connection.

Reliability is provided at the application layer through CRC-based error detection, Reed-Solomon forward error correction, and acknowledgement messages, allowing communication to continue even in such environments.

Design

The system relies primarily on Forward Error Correction (FEC) and proactive redundancy rather than retransmission-based reliability.

A message is split into `blocks`, which are then split further into `shards`.

Additionally, `parity` shards are also generated. Parity shards are redundant data and serves as a form of FEC (Forward error correction.).

Then, CRC checksums (Cyclic redundancy check; a math-based algorithm that generates checksum) are calculated and attached to every individual shards.

All shards are then send to the server and are verified by re-calculating CRC checksum. If the values does not match, then, a packet is deemed to be corrupted and discarded.

Any missing or discarded shards can then be reconstructed using the Reed-Solomon parity information, provided that a sufficient number of valid shards remain available.

Finally, the reconstructed shards are reassembled into their original blocks, and the blocks are combined to recreate the original message.

Packet Structure

The protocol uses a custom packet format to allow message reconstruction, error detection, and loss recovery over UDP.

Each packet contains both control information (metadata) and a portion of the original message payload.

- Message ID

Uniquely identifies a message transmission. This allows the receiver to distinguish between multiple messages that may be received concurrently.

- Block ID

Identifies which block of the original message a shard belongs to. This allows messages larger than a single block to be reconstructed correctly.

- Shard ID

Specifies the position of a shard within a block. This information is required for Reed-Solomon reconstruction and reassembly.

- Data / Parity Shard Counts

Indicates how many data shards and parity shards were generated for the block. This allows the receiver to determine reconstruction requirements.

- Payload Size

Stores the size of the data carried by the shard.

- CRC32

Stores the checksum calculated for the payload. The receiver recalculates this value to detect transmission errors.

- Payload Data

Contains either a data shard or a parity shard.

Protocol Workflow

The protocol follows a request-response communication model.

Client Side

1. User enters a message.
2. Message is assigned a unique Message ID.
3. Message is fragmented into blocks.
4. Each block is divided into data shards.
5. Reed-Solomon parity shards are generated.
6. CRC32 checksums are calculated.
7. All shards are transmitted to the server.
8. A timeout timer is started.

Server Side

1. Receive incoming shard packets.
2. Verify CRC32 checksum.
3. Discard corrupted packets.
4. Store valid shards.
5. Once sufficient shards are available, perform Reed-Solomon reconstruction.
6. Reassemble blocks into the original message.
7. Send an ACK or FAILURE response to the client.

Client Response Handling

Upon receiving a server response:

- **ACK** — Message was successfully reconstructed.
- **FAILURE** — Message could not be reconstructed.
- **TIMEOUT** — No response was received before the timeout period expired.